

AI and Machine Learning

- 작업기간 : 2021-07-21 ~ 2021-11-08

- 작업인원 : 4 명

- 작업 툴 : ipynb, Google Drive, Python (Scikit-learn, Keras 등)

- 작품소개 : 본 프로젝트는 Machine Learning 기법을 활용하여 주어진 binary classification 문제를 해결하기 위한 신경망 모델을 구축하는 프로젝트입니다. 프로젝트 진행 과정은 Logistic Regression, Decision Tree, Neural Network 모델을 활용하여 예측 성능을 비교하고, 각 모델의 하이퍼파라미터 튜닝 및 최적화를 통해 모델의 정확도를 최대한 높였습니다.

Task 1. 데이터 준비

- 목표: 모델 학습을 위한 데이터를 준비하고 전처리.
- 내용: 주어진 데이터를 로드하고, 필요한 열만 추출하여 X (특성)와 Y (타겟)을 나누었으며, 결측값 처리를 통해 데이터 정리.

Task 2. 로지스틱 회귀 모델 학습

- 목표: 로지스틱 회귀 모델을 학습하고 성능 평가.
- 결과: max_iter=2000으로 설정한 후, 학습 데이터에 대한 F1-score 89%, 정확도 87%를 기록. 테스트 데이터에서도 비슷한 성능을 보이며 과적합이나 과소적합 없이 잘 예측됨.

Task 3. 결정 트리 모델 학습

- 목표: 결정 트리 모델을 학습하고 최적의 하이퍼파라미터를 찾기.
- 결과: 교차 검증을 통해 모델의 정확도는 94-95%로 안정적이고, F1-score는 96-97%로 매우 높은 성능을 보였음.

Task 4. 신경망 모델 학습

- 목표: 간단한 신경망 모델을 학습하고 성능 평가.
- 구성: 2개의 은닉층과 1개의 출력층을 가진 신경망을 설계. 학습 후, 훈련 정확도 95%, 검증 정확도 96%를 기록하며 과적합 방지.
- 결과: 테스트 데이터에서 96% 정확도를 기록하며 높은 예측 성능을 보였음.

Task 5. 모델 비교 및 추천

- 목표: 각 모델의 성능을 비교하고 최적 모델 추천.
- 결과: 로지스틱 회귀 모델, 결정 트리 모델, 신경망 모델을 비교한 결과, 신경망 모델이 96% 정확도로 가장 우수한 성능을 보였으며, 추천 모델로 선정됨.

각 모델은 예측 정확도가 매우 높았으며, 과적합이나 과소적합 없이 안정적인 성능을 보여주었습니다. 이 프로젝트는 고급 머신러닝 기법을 활용한 모델 성능 최적화 및 성능 비교의 좋은 예시로, 실제 데이터를 기반으로 한 모델 학습과 최적화 과정에 대한 통찰을 제공하였습니다.

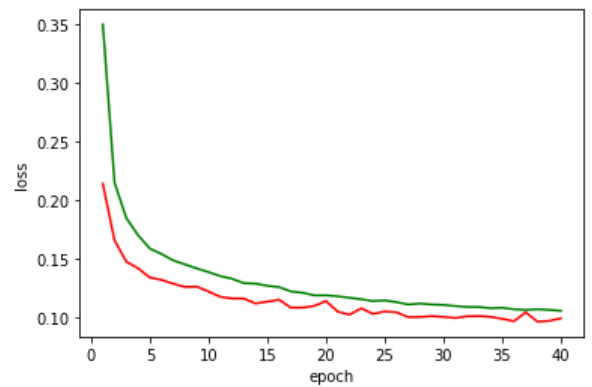
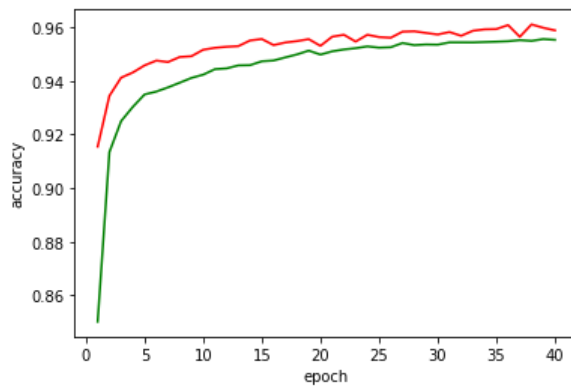
모델 구조도

```
def get_ff_model():
    model = Sequential()
    # Hidden layer 1
    model.add(Dense(50, input_shape=(25,))) #X train shape
    model.add(Activation('relu'))
    model.add(Dropout(0.1))

    # Hidden layer 2
    model.add(Dense(50))
    model.add(Activation('relu'))
    model.add(Dropout(0.1))

    # Output layer
    model.add(Dense(1))
    model.add(Activation('sigmoid'))
    return model
model_NN= get_ff_model()
```

모델 학습 그래프



분류 리포트

	precision	recall	f1-score	support
0	0.95	0.98	0.96	58697
1	0.97	0.94	0.95	44897
accuracy			0.96	103594
macro avg	0.96	0.96	0.96	103594
weighted avg	0.96	0.96	0.96	103594

Swift

- 작업기간 : 2021-07-30 ~ 2021-11-07
- 작업인원 : 1 명
- 작업 툴 : Swift, Xcode
- 작품 소개: Xcode를 사용하여 Swift로 애플리케이션을 개발하였으며, 주어진 과제를 해결하기 위한 기능을 구현하였다. 사용자 친화적인 인터페이스를 설계하고, 안정적인 성능을 제공하도록 최적화하는 데 중점을 두었다. 또한, 코드의 가독성과 유지보수성을 고려하여 효율적인 구조로 작성하였다.

Assignment 1: Kilowatts to Horsepower Converter

킬로와트(kW)에서 마력(hp)으로 변환하는 공식은 간단하지만, 마력 단위에는 세 가지 다른 표준이 존재한다. 이 프로젝트의 목표는 각 마력 단위와 kW 간 변환을 쉽고 빠르게 수행할 수 있는 앱을 개발하는 것이다. 또한, 마력에서 kW로 변환하는 기능도 포함할 예정이다.

이 프로젝트는 단순한 기능 구현을 넘어 Apple App Store에 출시할 수 있을 정도의 완성도 높은 앱을 만드는 것이 핵심 목표이다. 이를 위해 사용자의 편의성을 고려한 깔끔한 UI 디자인과 직관적인 UX를 적용할 예정이다.

Example1:

The screenshot shows a mobile app interface for converting kilowatts to horsepower. At the top, there is a text input field labeled "Enter kilowatts:" containing the value "528.9" and a unit label "kW". Below the input field are three buttons: "Convert" (with a circular arrow icon), "Reset" (with an 'x' icon), and "Swap" (with a double-headed arrow icon). Below the buttons, there are three rows of output results, each with a label, a value in a green box, and a unit label: "Result in mechanical horsepower:" with value "709.26658279" and unit "hp(I)", "Result in electrical horsepower:" with value "708.98123324" and unit "hp(E)", and "Result in metric horsepower:" with value "719.10387339" and unit "hp(M)".

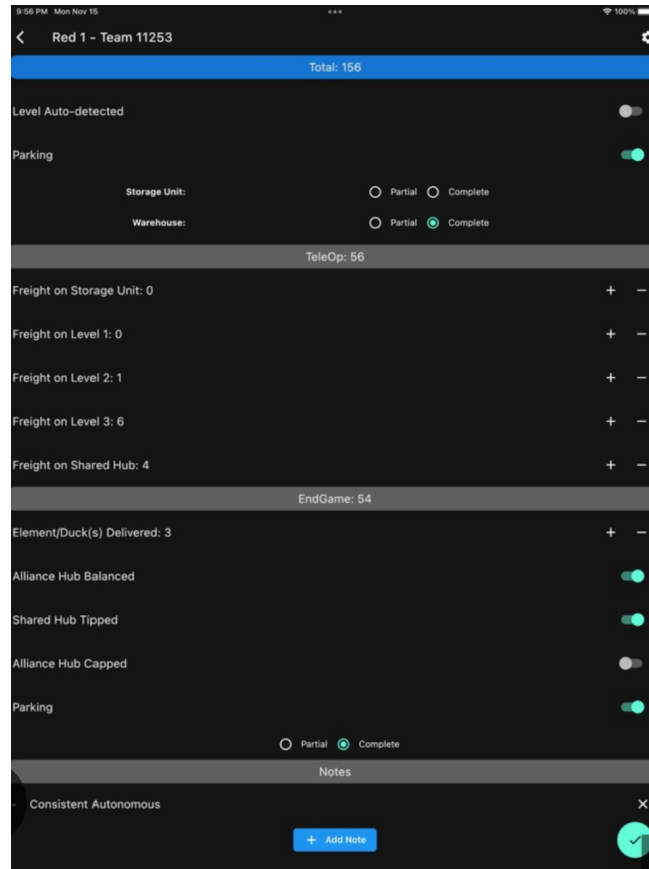
Assignment 2: Freight Frenzy Scoring App

FIRST Tech Challenge는 매년 6학년~12학년 학생들을 대상으로 진행되는 국제 로봇 대회로, 9월부터 4월까지 진행된다. 참가 팀들은 매년 변경되는 규칙에 따라 12" x 12" 경기장에서 대결하며, COVID-19 이후 온라인 방식의 "at-home" 대회도 운영되고 있다.

공식 경기에서는 지정된 채점 시스템을 사용하지만, 연습 경기에서는 별도의 점수 계산이 필요하다. 기존의 점수 계산 앱들은 기능이 부족하거나 채점 오류가 많아 사용이 불편한 경우가 많다. 따라서, 정확하고 사용하기 편리한 점수 계산 앱을 개발하는 것이 이 프로젝트의 목표이다.

이 앱은 직관적인 UI, 정확한 점수 계산 기능, 다양한 경기 방식 지원 등을 포함하여 실용성을 높일 예정이다.

Example2:



📌 프로젝트 GitHub: <https://github.com/Gitaebaechoas/portfolio/tree/main/Swift>

System Design Implementation

- 작업기간 : 2021-07-30 ~ 2021-11-12
- 작업인원 : 4 명
- 작업 툴 : Idle
- 작품소개 : 처음에 4명이 그룹으로 ****PMS 시스템(Assignment 1)****과 ****질문 답변 시스템(Assignment 2)****의 Volatility 개념을 조사하고 설계한 후 보고서를 작성하였다. PMS 시스템은 차량 번호판 인식을 통해 주차 위반 차량을 감지하는 시스템이며, 질문 답변 시스템은 사용자의 질문과 유사한 질문을 찾아 답변을 제공하는 기능을 갖춘다. 이후, ****질문 답변 시스템(Assignment 2)****의 기능을 파이썬을 이용해 직접 구현하였다.

Assignment 1: 주차 모니터링 시스템(PMS)

주차 모니터링 시스템(PMS) 개요

이 과제에서는 주차 모니터링 시스템(Parking Monitoring System, PMS)의 설계를 개발해야 한다. PMS는 차량 번호판 인식(LPR) 카메라와 GPS 기술을 활용하여 특정 구역의 차량 주차 상태를 모니터링하는 시스템이다. 주차 허가 없이 주차된 차량을 감지하고, 이를 기반으로 과태료를 부과하는 기능을 수행한다.

주요 기능 (Use Cases)

1. 차량 등록 (UC01)

- 운전자는 차량 번호판을 등록하고, 주차 위치 및 시간을 설정한 후 신용카드로 결제할 수 있다.
- 해당 기능은 모바일 앱 또는 주차장 주변의 PMS 키오스크에서 수행할 수 있다.

2. 차량 모니터링 (UC02)

- PMS 차량이 일정한 주기로 지정된 주차장을 순찰하며 LPR 카메라를 이용해 차량의 번호판을 인식하고, 주차된 위치와 시간을 기록한다.

3. 과태료 부과 (UC03)

- PMS 시스템이 유효한 주차 허가 없이 주차된 차량을 감지하면 PMS 차량의 운전자(주차 관리자)에게 경고한다.
- 관리자는 차량 번호판이 정확히 인식되었는지 확인한 후 과태료를 부과하고, 이를 시스템에 기록하며 종이 영수증을 인쇄해 차량 유리에 부착한다.

4. 실시간 통계 및 모니터링 (UC04)

- PMS 운영자는 특정 시간별 주차 차량 수, 발급된 과태료, PMS 차량 이동 경로 등의 데이터를 대시보드를 통해 확인할 수 있다.
- 해당 정보는 최소 5분마다 갱신된다.

시스템 요구사항

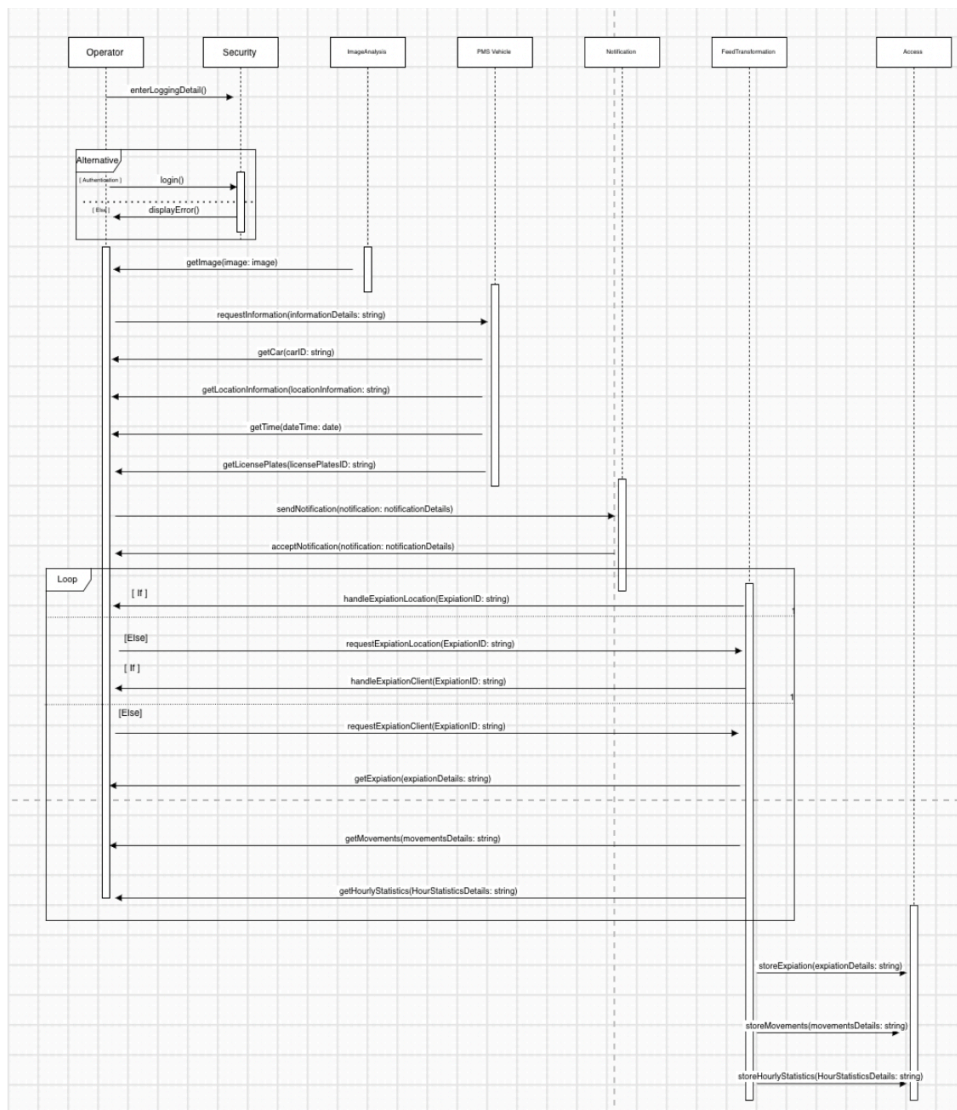
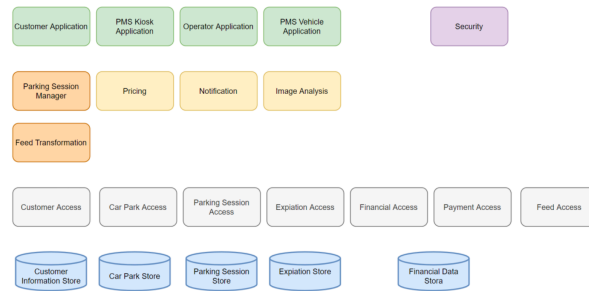
- **사용성(Usability):** 주차 관리자가 사용하는 인터페이스는 태블릿 환경에서 사용 가능해야 하며, 야외에서도 가독성이 좋아야 한다.
- **성능(Performance):**
 - 차량 속도가 10km/h 이하일 경우, 시스템은 5m 거리에서 0.5초 이내에 번호판을 인식할 수 있어야 한다.
- **확장성(Scalability):**
 - 10만 건 이상의 과태료 및 차량 사진 데이터를 저장할 수 있어야 한다.
 - 최소 100대의 PMS 차량을 동시에 지원할 수 있어야 한다.
- **비즈니스 규칙(Business Rules):**
 - 과태료 관련 차량 사진, GPS 위치, 타임스탬프는 최소 5년간 보관해야 한다.
 - 차량이 주차장에 진입한 후 10분 이내에 주차 허가를 받아야 하며, 이를 초과하면 과태료가 부과될 수 있다.

미래 확장성 (Future Evolutions)

- 현재 차량에 탑재된 이동형 LPR 카메라를 사용하고 있지만, 향후 주차장 입출구에 고정형 LPR 카메라를 설치하는 방안을 고려하고 있다.

- 고정형 카메라는 인식률이 더 높지만, 일부 모델은 클라우드 기반의 LPR 알고리즘을 사용하여 높은 네트워크 대역폭이 필요할 수 있다.

Component Decomposition



Assignment 2: 질문 답변 시스템 개요

질문 답변 시스템 개요

이 과제에서는 사용자가 자연어로 입력한 질문을 분석하고, 저장된 질문-답변 데이터베이스에서 가장 유사한 질문을 찾아 적절한 답변을 제공하는 간단한 질문 답변 시스템을 개발해야 한다.

주요 기능 (Use Cases)

1. 시스템 시작 (UC1 - Welcome)

- 시스템이 실행되면 사용자에게 환영 메시지를 표시한다.

2. 질문 입력 및 응답 (UC2 - Ask Question)

- 사용자는 한 문장으로 구성된 질문을 입력한다.
- 시스템은 입력된 질문을 저장하고, 기존의 질문 데이터베이스에서 가장 유사한 질문을 찾는다.
- 해당 질문과 연결된 답변을 찾아 사용자에게 제공한다.
- 이 과정은 사용자가 종료 명령을 내릴 때까지 반복된다.

3. 시스템 종료 (UC3 - Quit)

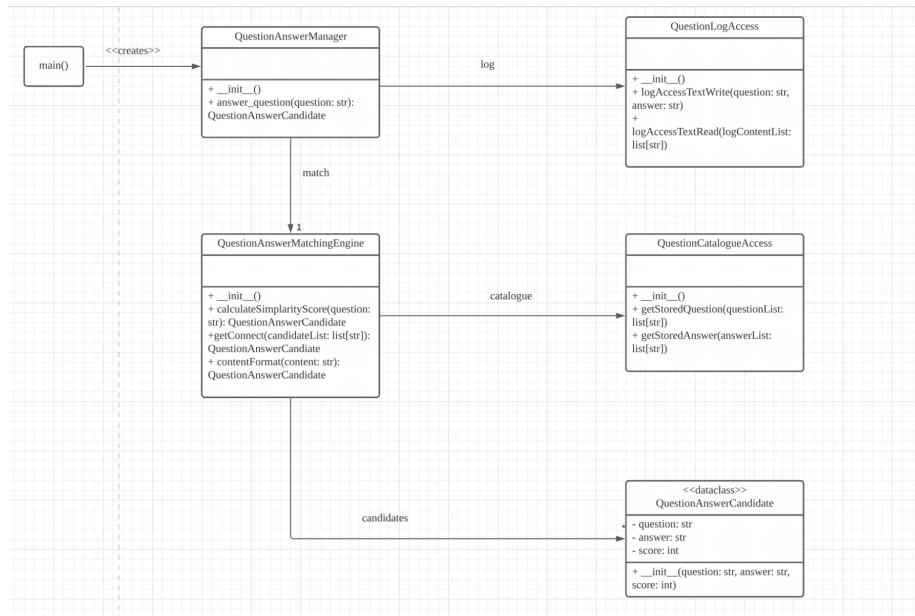
- 사용자가 빈 줄(공백)을 입력하면 시스템은 종료 메시지를 표시한 후 실행을 종료한다.

Example:

```
[
{
"question": "What day is today?",
"answer": "Monday"
},
{
"question": "What is the weather like today?",
"answer": "Same as yesterday."
}
]
```

시스템 개요 및 활용

이 시스템은 자연어 입력을 처리하여 의미적으로 유사한 질문을 찾아 자동으로 답변을 제공하는 기능을 수행한다. 단순한 매칭 방식으로 동작하며, 향후 발전된 AI 기반 질의응답 시스템으로 확장될 가능성이 있다.



📌 프로젝트 GitHub: <https://github.com/Gitaebaechaos/portfolio/tree/main/System Design Implementation>

System Design Studio

- 작업기간 : 2021-07-30 ~ 2021-11-20

- 작업인원 : 6 명

- 작업 툴 : Word

- 작품소개 : 이 프로젝트는 6명이 한 팀이 되어 응급차 출동 시스템을 설계하는 작업이었다. 팀원들은 시스템을 여러 부분으로 나누어 조사하고, 다이어그램 및 인터페이스를 제작하며, 최종적으로 시스템 디자인을 완성했다. 시스템은 000 응급 전화를 접수하고 환자의 상태를 분류하여 적절한 구급차를 배치하는 역할을 한다. 또한, 구급대원이 실시간으로 출동 지시를 받고 병원 이송까지 원활하게 진행할 수 있도록 지원한다.

응급차 긴급 출동 시스템(Ambulance Emergency Response System) 요약

이 시스템은 SA Ambulance의 요청에 따라 000(한국의 119)로 접수된 응급 전화를 처리하고, 환자의 상태를 분류하여 적절한 구급차를 배치하는 역할을 한다.

1. 응급 전화 접수 및 분류 (Triage)

- 000으로 걸려온 응급 전화를 중앙 콜센터에서 접수하여 분류한다.
- 콜센터 직원(Call Taker)은 의료 지식 없이 시스템이 제공하는 결정 트리(If-Then 방식 가이드라인)를 따라 환자의 상태를 1~5단계로 분류한다.

- 1단계(최고 긴급, 예: 심정지) 환자는 8분 이내에 출동해야 하며, 2단계(중증 가슴 통증 등)는 18분 이내에 출동해야 한다.
- 허위 신고나 장난 전화도 시스템에 기록되지만, 출동 케이스로 분류되지는 않는다.
- 시스템은 전화번호 정보를 활용해 대략적인 신고 위치를 자동으로 식별한다.

2. 구급차 배치 (Dispatch)

- 콜센터에서 분류된 응급 상황은 해당 지역을 담당하는 디스패처(Dispatcher)에게 전달된다.
- 디스패처는 사건의 긴급도와 가장 가까운 구급차 위치를 고려하여 적절한 구급차를 배치한다.
- 만약 배치 가능한 구급차가 없을 경우, 다른 구급대원에게 출동 가능 여부를 요청한다.
- 디스패처 본부는 지역별로 운영되며, 각 본부는 평균적으로 20개 구급차 팀을 관리한다.

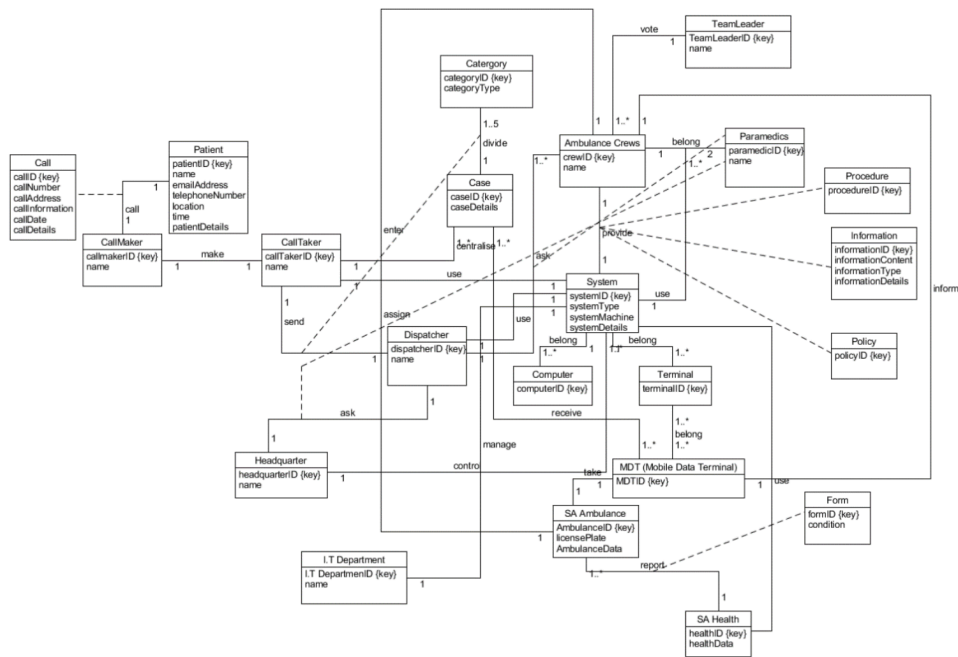
3. 구급차 출동 및 응급 대응 (Response)

- 두 명의 구급대원이 한 팀을 이루어 구급차를 운영하며, 차량 내 모바일 데이터 터미널(MDT)을 통해 출동 지시를 받는다.
- 구급대원은 출동 승인, 이동 시작, 현장 도착 등을 시스템에 기록하며, 필요한 경우 환자를 병원으로 이송한다.
- 카테고리 1(최고 긴급)의 경우 사이렌과 경광등을 켜고 이동한다.
- 시스템은 병원의 수용 가능 여부, 교통 상황, 장비 수리 여부 등을 반영하여 최적의 병원을 추천한다.
- 응급 상황에 따른 최신 정책 및 절차(예: COVID-19 대응 지침)를 실시간으로 제공한다.
- 출동 및 주요 사건의 날짜와 시간은 법적 요구 사항에 따라 영구적으로 기록된다.

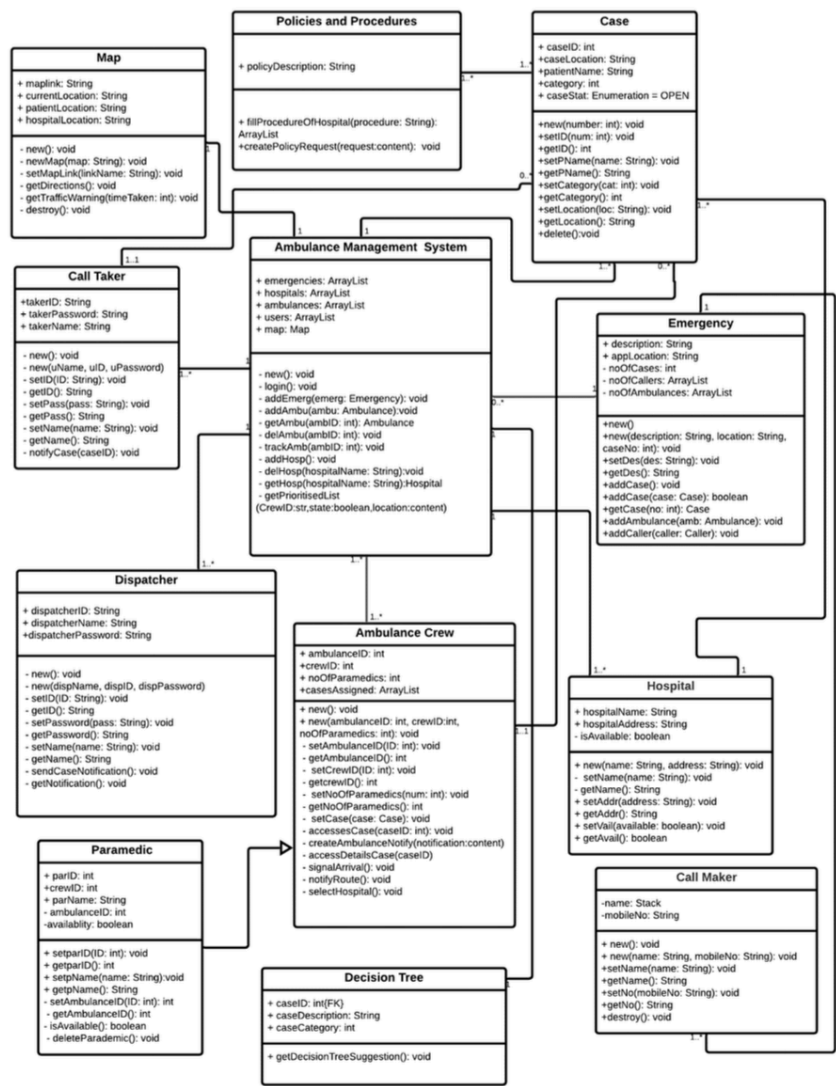
4. 운영 및 보고

- 구급대원은 24시간 교대 근무를 하며, 팀 리더가 각 근무조(A, B, C, D)를 관리한다.
- 팀 리더는 근무 중 구급대원의 출동 기록 및 피로도를 분석하는 보고서를 확인할 수 있다.
- 시스템은 사용자의 피로도를 고려한 인터페이스와 기능을 제공하여 실수를 최소화한다.

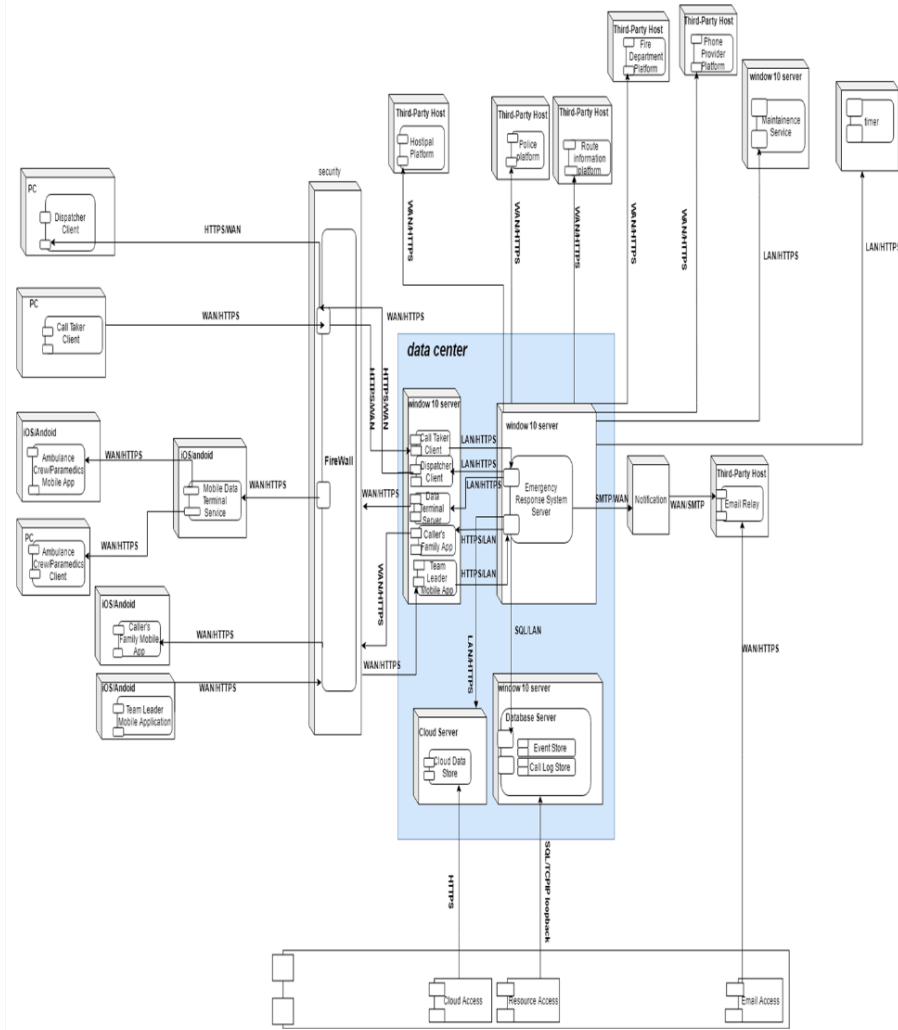
Domain Model Class Diagram



UML Class Diagram



9. Deployment Diagram



📌 프로젝트 GitHub: [https://github.com/Gitaebaechaos/portfolio/tree/main/System Design Studio](https://github.com/Gitaebaechaos/portfolio/tree/main/System%20Design%20Studio)

Big Data

- 작업기간 : 2022-02-28 ~ 2022-06-03
- 작업인원 : 1 명
- 작업 툴 : Word
- 작품소개 : 이 작품은 빅데이터와 인공지능(AI) 기술을 활용하여 특정 산업과 조직의 비즈니스 요구에 맞는 기술 솔루션과 전략을 제안하는 내용을 다룹니다. 과제 1에서는 AI 기술을 통해 산업별 문제 해결 방안을 제시하고, 과제 2에서는 빅데이터 전략을 통해 조직의 데이터 활용도를 높이는 방안을 제안합니다. 이를 통해 각 조직이 데이터 기반 의사결정을 할 수 있도록 돕고, 실용적이고 실행 가능한 솔루션을 제공합니다.

Assignment 1: 인공지능(AI) 데이터 기술 솔루션

특정 산업의 조직들을 대상으로, 그들이 필요한 비즈니스 요구사항에 맞는 인공지능(AI) 데이터 기술 솔루션을 제안하는 보고서를 작성.

핵심 사항:

- 산업에 대한 이해: 제안할 기술이 어떤 산업에 적용될 수 있는지, 해당 산업의 비즈니스 요구를 어떻게 해결할 수 있는지를 이해하고 설명해야 한다.
- 기술 설명: 제안하는 AI 기술이나 기술적 기법을 비즈니스용 관점에서 쉽게 설명해야 하며, 기술적인 용어를 알기 쉽게 풀어서 설명.
- 증거 기반 제시: 제안한 기술의 효과를 입증할 수 있는 실증적인 데이터나 사례를 통해 설명해야 한다.
- 목표: 투자 결정을 내릴 수 있도록 비즈니스적인 관점에서 설득력 있는 보고서를 작성.

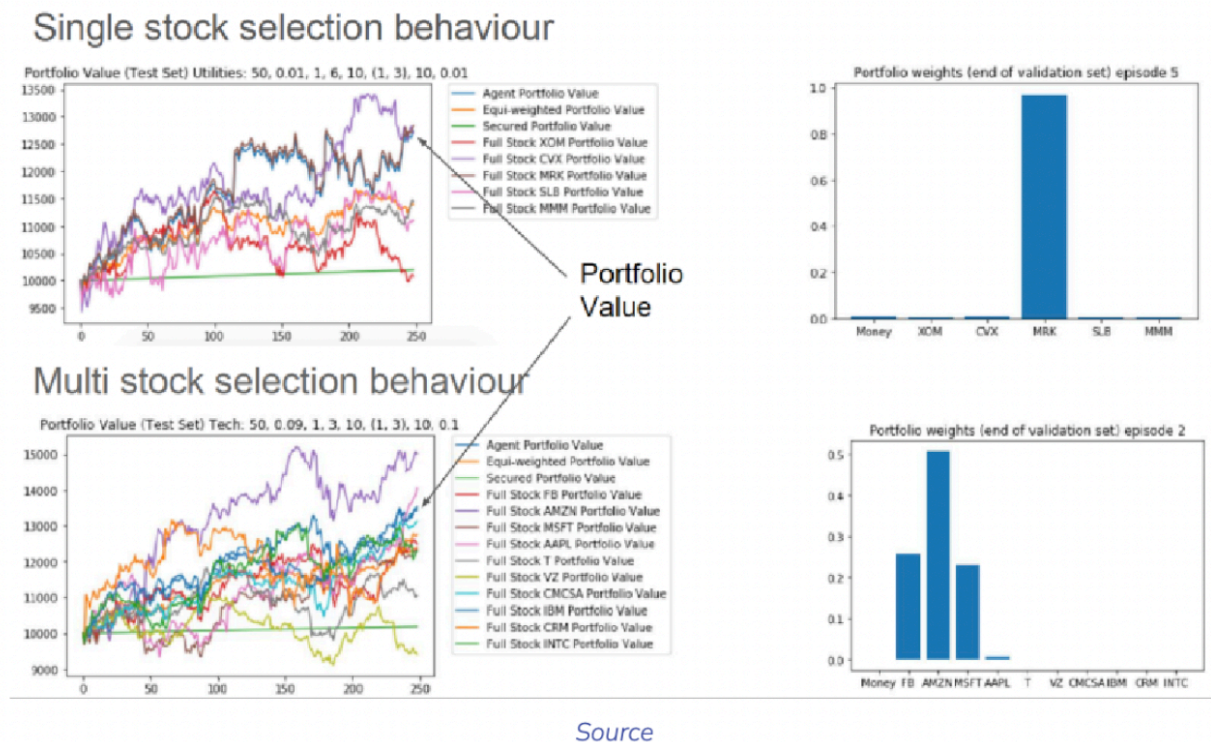


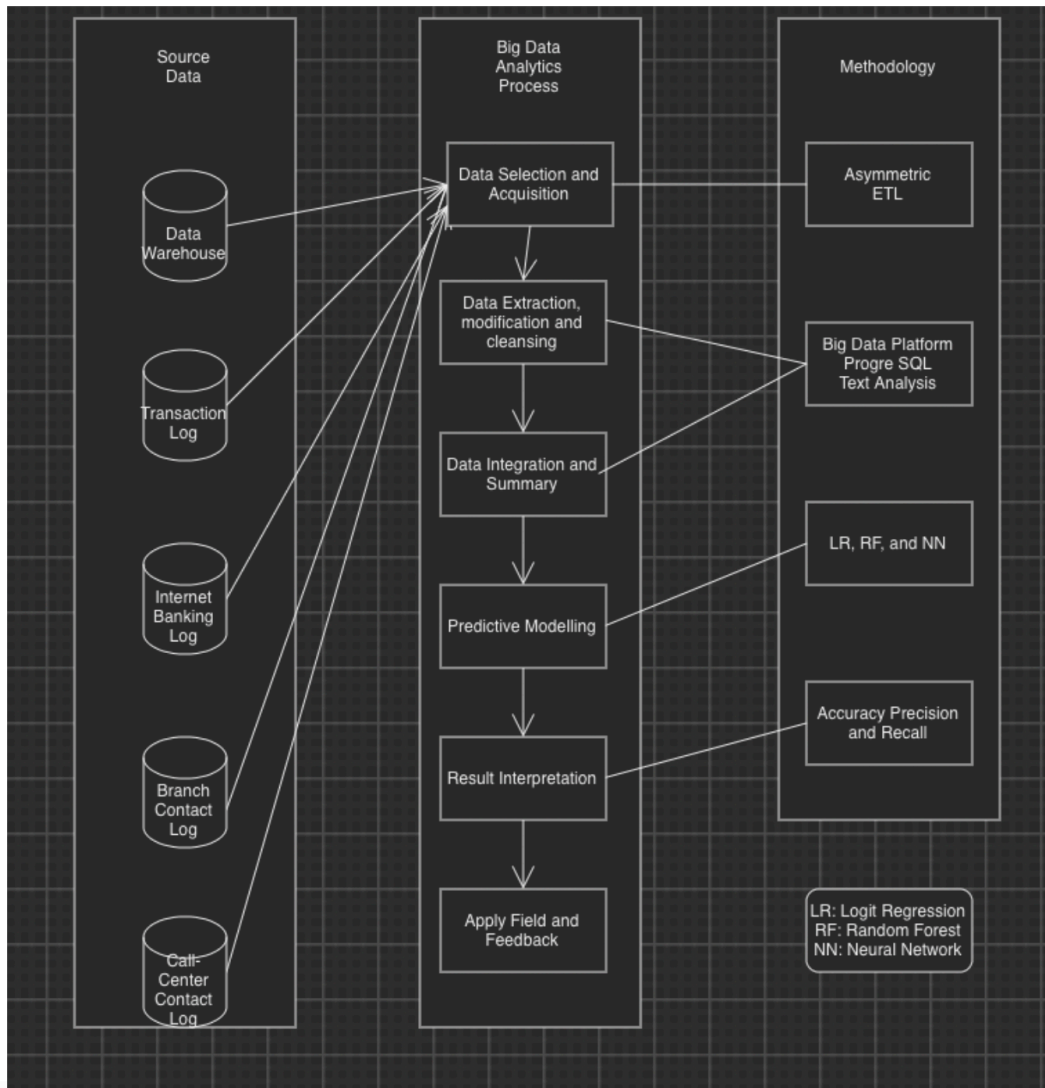
Figure 2. Portfolio Value Graph (Amrouni, Moulin & Mizrahi 2022)

Assignment 2: 비즈니스 우선순위에 맞는 빅데이터 전략

특정 조직을 대상으로, 해당 조직의 비즈니스 우선순위에 맞는 빅데이터 전략을 제안하는 프로포절 작성.

핵심 사항:

- 기술 제안: 빅데이터 기술 및 분석 툴, 최종 사용자 애플리케이션 등을 제안해야 하며, 각 기술의 장점과 필요성을 명확히 설명해야 한다.
- 하이 레벨 아키텍처 다이어그램: 제안한 기술들이 어떻게 통합되어 동작할 수 있는지 시각적으로 표현해야 한다.
- 증거 기반 제시: 제안한 빅데이터 기술이 실제 비즈니스 문제를 해결할 수 있다는 것을 증명할 수 있는 사례나 데이터를 바탕으로 논리를 전개.
- 목표: 빅데이터를 잘 활용하지 못하고 있는 조직이 어떻게 데이터를 더 잘 활용할 수 있을지에 대한 전략적 제안을 하여, 고위 경영진이 실질적인 결정을 내릴 수 있도록 도와야 한다.



📌 프로젝트 GitHub: <https://github.com/Gitaebaechaos/portfolio/tree/main/BigData>

Data Structure

- 작업기간 : 2022-03-07 ~ 2022-07-09

- 작업인원 : 1 명

- 작업 툴 : java, eclipse

- 작품소개 : SimFarm은 농장 관리 시뮬레이션 게임으로, 다양한 작물을 재배하고 관리하는 시스템을 구현하였다. GameStock은 기본적인 주식 시장 시뮬레이션 시스템으로, 주식 거래소, 중개인, 거래 처리 시스템을 Java로 개발하였다. 두 프로젝트 모두 객체지향 프로그래밍을 활용하여 시스템의 핵심 로직과 데이터 흐름을 구조적으로 설계하였다. 이를 통해 게임 내 경제 시스템과 거래 메커니즘을 효과적으로 모델링하였다.

Assignment 1: Farm 시뮬레이션 게임 개요

이 프로젝트는 농장을 운영하는 시뮬레이션 게임으로, 플레이어가 밭을 관리하며 곡물과 과일을 재배하고 수확하며 돈을 벌어나가는 구조이다.

1. 아이템 시스템

게임의 모든 요소는 **Item** 클래스를 기반으로 하며, 나이(Age), 성숙 나이(Maturation Age), 죽는 나이(Death Age), 금전적 가치(Value)를 가진다.

- **Item** 클래스는 직접 인스턴스화할 수 없으며, 하위 클래스로 확장된다.
- **Food**(음식) → 직접 생성할 수 없고, **Grain**(곡물)과 **Apple**(사과)로 세분화됨.
 - **Grain**(곡물): 2턴 후 성숙($G \rightarrow g$), 6턴 후 죽음, 수확 시 가치 2.
 - **Apple**(사과): 3턴 후 성숙($A \rightarrow a$), 5턴 후 죽음, 수확 시 가치 3.
- ****잡초(Weed)**, 경작되지 않은 땅(**UntilledSoil**), 경작된 땅(**Soil**)도 아이템으로 분류됨.
 - 잡초와 경작되지 않은 땅은 절대 사라지지 않으며, 경작된 땅(**Soil**)은 가치가 0.

2. 밭(Field) 시스템

밭은 동적으로 크기가 설정되는 **2D** 배열의 **Item** 객체로 구성된다.

- 처음 생성될 때 ****모든 칸이 Soil(경작된 땅)****으로 설정됨.
- 매 턴이 지나면(**tick()** 호출),
 - 모든 아이템의 나이가 증가
 - 일정 확률(**20%**)로 잡초가 자람
 - 죽은 작물은 경작되지 않은 땅(**UntilledSoil**)으로 변경됨
- 주요 기능:
 - **till(x, y)** → 특정 좌표를 경작하여 Soil로 변경
 - **plant(x, y, item)** → 특정 좌표에 아이템 심기
 - **getSummary()** → 현재 밭의 작물 수량 및 전체 가치 출력

3. 농장(Farm) 시스템

플레이어는 특정 자금을 가지고 시작하며, 밭을 관리하고 작물을 키워 돈을 버는 목표를 가진다.

- 매 턴마다 필드 상태가 업데이트되며, 플레이어는 다양한 액션을 수행할 수 있다.
- 사용 가능한 명령어:
 - **t x y** → (경작) 땅을 갈아 Soil로 변경

- **h x y** → (수확) 성숙한 작물 수확하여 돈 획득
- **p x y** → (심기) 사과 또는 곡물 심기 (잔고 부족 시 실패)
- **s** → (요약) 현재 필드 상태 출력
- **w** → (대기) 아무 행동 없이 시간 경과
- **q** → (종료) 게임 종료

플레이어는 수익을 창출하며 농장을 운영하고, 효율적인 작물 재배 전략을 통해 더 큰 이익을 목표로 한다. 게임은 턴 방식으로 진행되며, 밭과 자금 상태가 실시간으로 변화하는 시뮬레이션 요소를 포함한다.

Output:

```

 1 2 3 4 5 6 7 8 9 10
1 . . . . . A . . . . .
2 . . . . . . . . . .
3 . . . . . . . G . .
4 . . . . . . . . . .
5 . a . . . . . . . .
Bank balance: $10

Enter your next action:
t x y: till
h x y: harvest
p x y: plant
s: field summary
w: wait
q: quit
h 5 1
Sold 'A' for 0

 1 2 3 4 5 6 7 8 9 10
1 . . . . . . . . . .
2 . . . . . . . . . .
3 . . . . . . . G . .
4 . . . . . . . . . .
5 . a . . . . . . . .
Bank balance: $10

Enter your next action:
t x y: till
h x y: harvest
p x y: plant
s: field summary
w: wait
q: quit
h 8 3
Sold 'G' for 2

 1 2 3 4 5 6 7 8 9 10
1 . . . . . . . . . .
2 . . . . . . . . #
3 . . . . . . . . . .
4 . . . . . . . . . .
5 . A . . . . . # . .
Bank balance: $12
Mail
```

Assignment 2: Farm 시뮬레이션 게임 개요

주식 시장 시뮬레이션 개요

이 프로젝트는 기본적인 주식 시장 시스템을 구현하는 과제로, 주식 거래소(Stock Exchange), 중개인(Broker), 거래(Trade), 상장 기업(Listed Company) 등의 요소를 포함한다.

1. 상장 기업 (Listed Company)

주식이 거래되는 기업으로, 다음과 같은 정보를 가진다.

- 회사명(Name) 및 회사 코드(Code) (약어)
- 현재 주가(Current Price)

2. 거래 (Trade)

주식을 사고팔기 위한 객체로, 다음 정보를 포함한다.

- 거래 수량(**Number of Shares**)
- 대상 기업(**Company**)
- 거래를 처리할 중개인(**Broker**)

3. 중개인 (Broker)

개별 사용자로부터 거래를 받아 큐(**Queue**) 형태로 저장하고 처리한다.

- 일반적으로 선입선출(**FIFO**) 방식으로 거래를 처리
- **추천 주식 목록(Watchlist)**을 보유하여 특정 기업의 주식을 추천
- 비윤리적 중개인은 자신이 이익을 얻기 위해 특정 거래를 큐의 뒤로 미루는 행위를 할 수도 있음

4. 주식 거래소 (Stock Exchange)

여러 중개인이 거래를 수행하는 장소로, 중개인의 큐에서 거래를 하나씩 처리하여 주식 시장을 운영한다.

- 각 중개인에게 다음 거래 요청 후 거래를 처리
- 거래가 성사되면 해당 기업의 주가가 변동됨
- 여러 차례 반복 수행하여 전체 시장의 거래를 처리

이 프로젝트는 거래의 공정성, 중개인의 역할, 주가 변동 등의 개념을 이해하고 구현하는 것이 핵심이며, 비윤리적 거래 행위 감지 등의 요소를 포함하여 실제 주식 시장의 일부 기능을 반영한다.

The screenshot displays an IDE with two main panels. The left panel shows the JUnit test results for a class named `ListTest`. The tests listed include `testRemoveTrade_End`, `testRemoveTrade_Middle`, `testAdd`, `testGet`, `testRemoveTrade_Single`, `testSize`, `testList_CopyConstructor`, `testAddIntTrade`, `testToString`, `testList_Node`, `testRemoveIntEnd`, `testContains`, `testRemoveTrade_Beginning`, `testRemoveIntBeginning`, `testIndexOf`, `testIsEmpty`, and `testRemoveInt`. All tests passed successfully. The right panel shows the Java code for `ListTest.java`, which is a JUnit test class for a linked list. The code includes package declarations, imports, and a `setUp` method that initializes a linked list with three nodes. The console at the bottom shows the output of the tests, indicating that all tests passed.

```
1 package unisa.dse.a1.junit;
2
3 import static org.junit.Assert.*;
4
5 /**
6  * @author james.walsh@unisa.edu.au
7  */
8 public class ListTest extends DSUnitTesting {
9
10     private static String ID = "unisa.dse.a1.junit.List";
11     Node chain1;
12     Node chain2;
13     Node chain3;
14
15     DSEList l;
16
17     @Before
18     public void setUp() {
19         {
20             chain1 = new Node(null, null, new Trade(null, 1));
21         }
22         {
23             Node c21 = new Node(null, null, new Trade(null, 1));
24             Node c22 = new Node(null, null, new Trade(null, 1));
25
26             chain2 = c21;
27             c21.next = c22; c22.prev = c21;
28         }
29         {
30             Node c31 = new Node(null, null, new Trade(null, 1));
31             Node c32 = new Node(null, null, new Trade(null, 2));
32             Node c33 = new Node(null, null, new Trade(null, 3));
33
34             chain3 = c31;
35         }
36     }
37 }
```

Operating Systems and Tool Chains

- 작업기간 : 2022-03-07 ~ 2022-07-10

- 작업인원 : 1 명

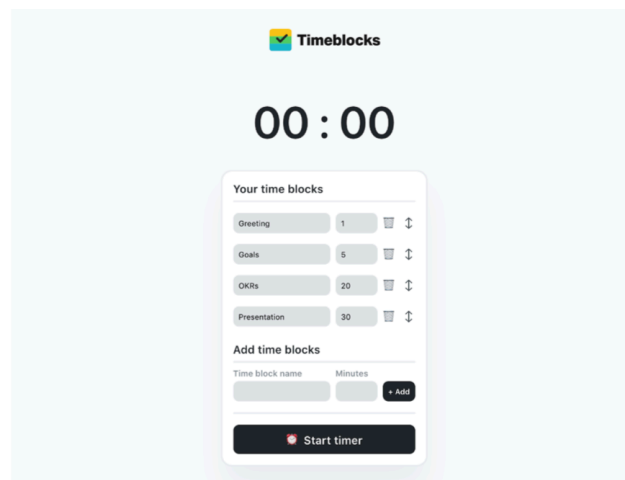
- 작업 툴 : Idle, word

- 작품소개 : 본 프로젝트는 팀 회의용 카운트다운 타이머와 운영체제 재설계를 개발한 것이다. 카운트다운 타이머는 회의 시간을 효율적으로 관리할 수 있도록 타이머와 경고 시스템을 구현했다. 또한, 운영체제 재설계 프로젝트에서는 프로세스 스케줄링, 파일 시스템, 메모리 관리의 최신 동향을 분석하고, 특정 사용 시나리오에 맞춰 효율적인 운영체제 구조를 제안하였다. 두 프로젝트 모두 성능 개선과 효율성을 목표로 하여 실용적인 시스템과 도구를 설계하였다.

Assignment 1: 팀 회의용 카운트다운 타이머 개발

본 프로젝트는 팀 회의를 효과적으로 관리하기 위한 카운트다운 타이머를 개발하는 것이다. 회의 시간이 초과되지 않도록 정해진 시간 동안 타이머를 실행하고 경고를 제공하는 기능을 구현하였다.

이를 통해 비즈니스 및 학습 환경에서 효율적인 시간 관리가 가능하도록 지원한다. 소프트웨어 개발의 체계적인 워크플로우를 적용하여 실용적인 도구를 제작하였다.



Assignment 2: 운영체제 및 툴체인 연구 및 재설계

본 프로젝트는 운영체제 내부 구조(프로세스 스케줄링, 파일 시스템, 메모리 관리)의 최신 동향을 조사하고, 특정 사용 시나리오에 맞춰 재설계하는 것이다.

각 개념을 분석하고, 현대적인 개발 환경에서의 효율성을 높일 수 있도록 최적화된 운영체제 구조를 제안하였다.

이를 통해 성능 개선과 시스템 자원 활용 극대화를 목표로 한 OS 설계를 수행하였다.

Category	Ext1	Ext2	Ext3	Ext4
Developer	Stephen Tweedie	R�my Card	Stephen Tweedie	Mingming Cao, Andreas Dilger, Alex Zhuravlev (Tomas), Dave Kleikamp, Theodore Ts'o, Eric Sandeen, Sam Naghshineh, others (Theodore Ts'o presented to make ext4
Date	04/1992	01/1993	11/2001	10/2006
Partition ID	-	Apple_UNIX_SVR2, 0x83(MBR), EBD0A0A2-89C5-4433-87C0-68B6B72699C7 (GPT)	0x83(MBR), EBD0A0A2-89C5-4433-87C0-68B6B72699C7 (GPT)	s0x83(MBR), EBD0A0A2-89C5-4433-87C0-68B6B72699C7 (GPT)
Directory Structure	-	Table	Table, Hashed B-tree(dir_index)	Linked List, Hashed B-tree
File Structure	Bitmap(free space), table(meta data)	Bitmap(free space), table(meta data)	Bitmap(free space), table(meta data)	Extents/Bitmap
Bad Block Structure	Table	Table	Table	Table
Maximum File size	-	16GB - 2TB	16GiB - 2TiB	16TiB (based on 4k blocks)
Maximum Number of File	-	10 to the 18th power	Variable, assigned at writing time	4 billion (saved at file system creation time)
Maximum File Name Length	-	255byte	255byte	256byte
Maximum Volume Size	-	2TB - 32TB	2TiB - 16TiB	1 EiB
Date Permissions	-	Modification(mtime), Characteristic Modification(ctime), Access(atime)	Modification(mtime), Characteristic Modification(ctime), Access(atime)	Modification(mtime), Characteristic Modification(ctime), Access(atime), Delete(dtime), Create(ctime)
Data Range	-	14/12/1901 ~ 18/01/2038	14/12/1901 ~ 18/01/2038	14/12/1901 ~ 25/04/2514
Data Accuracy	-	1s	1s	Nano second

Page 6 of 13

Gitae Bae baegy002(110310861)

File System Permissions	POSIX	POSIX	Unix permissions, ACLs, arbitrary security features (Linux 6.2 and later)	POSIX
Operating System	-	Linux, BSD, WINDOWS (through IFS), OSX(through IFS)	Linux, BSD, WINDOWS, OSX(through IFS)	Linux, WINDOWS (when using ext2fsd)
Characteristic	-	-	sNo-atime, append-only, synchronous-write, no-dump, h-tree(directory), immutable, journal, secure-delete, top(directory), allow-undelete	Extents, noextents, mballocc, nomballocc, delallocc, nodelallocc, data=journal, data=ordered, data=writeback, commit=nsec, orlov, oldallocc, user_xattr, nouser_xattr, acl, noacl, bsddfc, minixdf, bh, nobh, journal_dev

Table1: Summary of Ext at the file system

프로젝트 GitHub: <https://github.com/Gitaebaechaos/portfolio/tree/main/Operating System>

Booking System

- 작업기간 : 2022-06-27 ~ 2022-10-03
- 작업인원 : 3 명
- 작업 툴 : GitHub, xampp control panel, notepad
- 작품소개 : xampp control panel을 이용해서 Apache와 MySQL을 이용해서 notepad로 php코드를 작성을 하고 phpmyadmin으로 데이터를 관리하면서 레스토랑 북킹 로그인 시스템과 예약 시스템

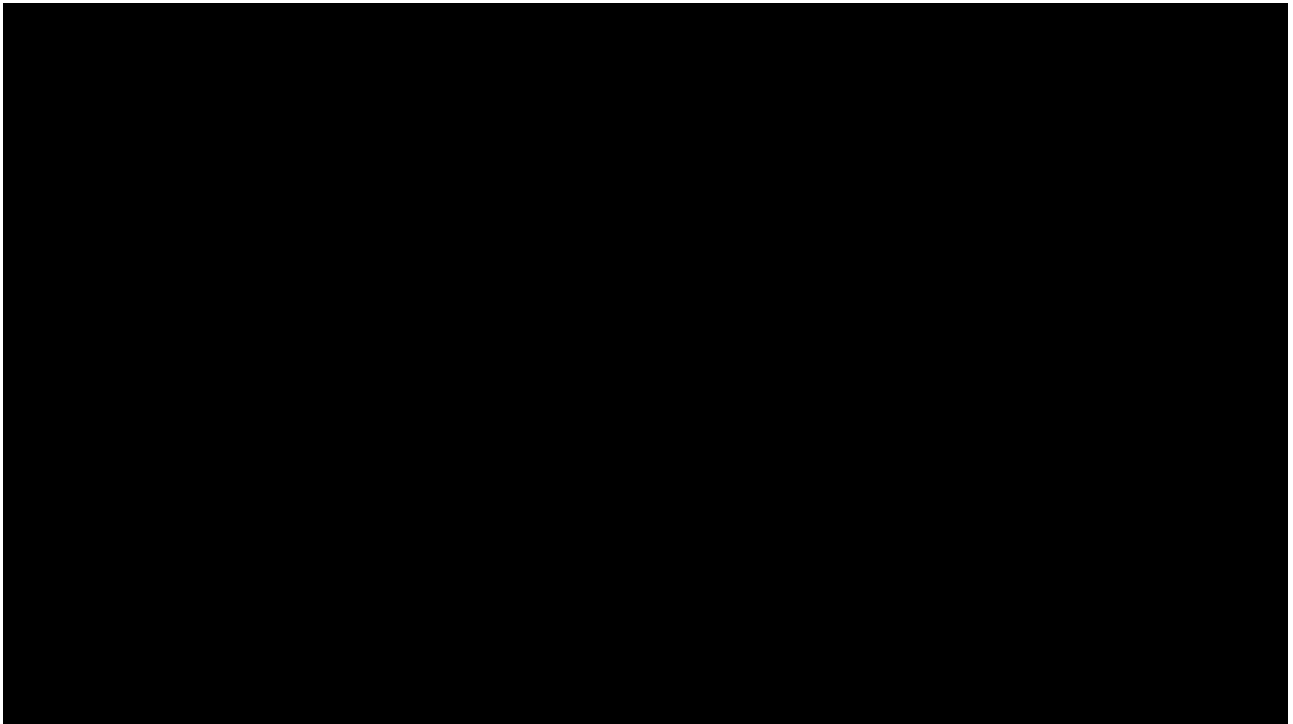
템 그리고 웹페이지를 따로 만들어서 데이터를 불러들이고 그걸 웹페이지에 전달하여 보이게 했던 파일이다.

레스토랑 예약 시스템은 사용자 로그인 및 회원 가입 기능을 제공하며, 사용자는 날짜별 예약을 통해 레스토랑 예약을 진행할 수 있습니다. 사용자는 예약 시 날짜와 시간을 선택하여 예약을 할 수 있으며, 해당 예약 정보는 **MySQL** 데이터베이스에 저장됩니다.

관리자는 **phpMyAdmin**을 통해 예약 데이터를 관리하고, 예약 현황을 실시간으로 확인할 수 있습니다.

웹 페이지는 사용자에게 예약을 진행할 수 있는 직관적인 인터페이스를 제공하고, 사용자가 입력한 예약 정보를 서버로 전달하여 예약 내용을 보여줍니다. 이를 통해 데이터베이스와 연동된 동적인 웹 시스템을 구현하고, 실시간 예약 관리가 가능하도록 했습니다.

비디오:



 프로젝트 GitHub: https://github.com/Gitaebaechaos/portfolio/tree/main/오케오탁시스템_트케모

Agile and Governance

- 작업기간 : 2022-07-04 ~ 2022-10-03

- 작업인원 : 9 명
- 작업 툴 : Atlassian, jira
- 작품소개 : 다양한 인원으로 다양한 인종끼리 협심하여 agile 과 governance로 atlassian 사이트로 jira를 이용해서 매주 그룹원들과 어느 정도 작업을 하며 어느정도 시간이 걸렸는지 기록하며 어떤 주제가 주어진다면 토론하며 조사해서 프리젠테이션을 했으며 각자 조사를 해서 모은 정보를 바탕으로 리포트를 만들었다.

이 문서는 ITIL(정보 기술 인프라 라이브러리) 평가 보고서를 작성하는 데 필요한 지침을 제공합니다. 이 보고서는 주어진 사례를 기반으로 ITIL의 핵심 개념, 프로세스 및 실습을 분석하는 내용으로 구성됩니다. 보고서의 목표는 실제 비즈니스 환경에서 ITIL을 어떻게 적용할 수 있는지 설명하는 것입니다. 이 보고서를 제출하기 전에는 불필요한 안내 텍스트와 평가 기준을 삭제해야 하며, 최종 제출된 보고서는 UniSA STEM에서 요구하는 형식을 반영해야 합니다.

PIEDOD Heatmap:

ITIL Practice	P	I	E	D	O	D
Information security management	3	3	3	3	3	3
Relationship management	3	3	3	3	3	3
Supplier management	2	2	3	3	3	3
IT asset management	3	2	2	3	3	3
Monitoring and event management	1	2	2	3	2	3
Release management	2	1	1	3	3	2
Service configuration management	2	2	2	3	3	3
Deployment management	1	2	1	3	3	0
Continual improvement	3	3	3	3	3	3
Change enablement	3	3	3	3	3	3
Incident management	1	2	3	3	3	3
Problem management	2	3	2	2	2	3
Service request management	1	2	3	3	3	3
Service desk	1	3	3	3	2	3
Service level management	3	3	3	3	3	3

📌 프로젝트 GitHub: <https://github.com/Gitaebaechaos/portfolio/tree/main/Agile and Governance>

C++

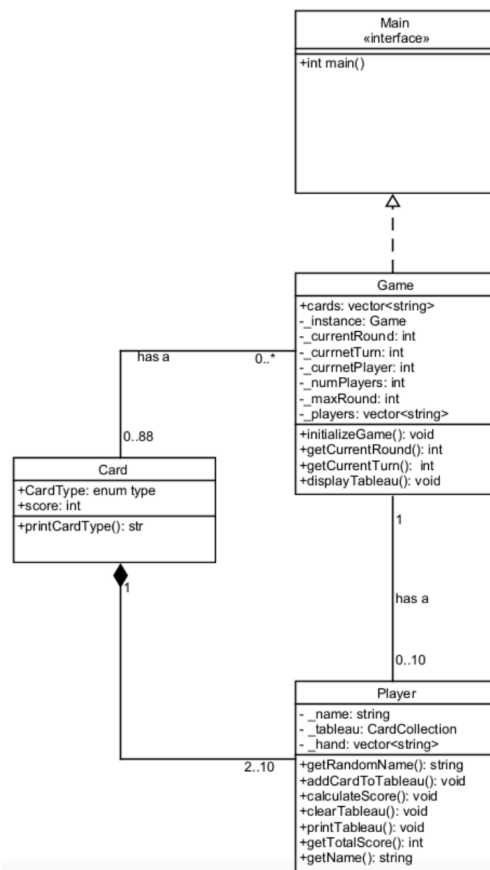
- 작업기간 : 2023-03-08 ~ 2023-07-05
- 작업인원 : 1 명
- 작업 툴 : C++
- 작품소개 : "Sushi Slam!"은 C++을 사용해 개발한 카드 드래프팅 게임으로, 두 명의 플레이어가 스시 카드를 선택하여 점수를 얻고 경쟁하는 게임입니다. "Sushi Go!"를 기반으로 하여 독창적인 규칙을 적용하였습니다. 또한, "환자 관리 시스템"은 환자의 생체 신호를 기록하고 질병에 따라

경고 레벨을 제공하는 시스템으로, C++을 이용해 환자 정보를 관리하고 경고 시스템을 구현하였습니다. 두 시스템 모두 실시간 데이터 처리와 사용자 상호작용을 중점으로 설계되었습니다.

Assignment 1: Sushi Slam! 게임 개발

이 과제는 C++을 사용하여 "Sushi Slam!"이라는 커맨드라인 기반 카드 드래프팅 게임을 개발하는 과제입니다. 게임은 두 명의 플레이어가 각각 카드 한 장을 선택하여 점수를 얻고, 세 번의 라운드를 통해 더 많은 점수를 획득하는 방식입니다. 게임에서 카드는 여러 종류의 스시를 나타내며, 각 스시의 종류는 특정한 점수 규칙을 가지고 있습니다. 이 게임은 "Sushi Go!"와 유사하지만 독창적인 방식으로 구현되었습니다.

Class Diagram



Output:

```

SUSHI SLAM!

~~~ round 1/3 ~~~
PLAYER Sasha TURN
Tableau:
Current hand:
1. Nigiri (Squid)
2. MakiRoll (1)
3. Nigiri (Salmon)
4. Sashimi
5. Tempura
6. Sashimi
7. Dumpling
8. Dumpling
9. MakiRoll (2)
10. MakiRoll (3)
Select a card to add to your tableau:
    
```

Assignment 2: 환자 관리 시스템 개발

이 과제는 환자의 생체 신호(체온, 혈압, 심박수, 호흡수)를 기록하고, 질병에 따라 적절한 경고 수준을 제공하는 환자 관리 시스템을 개발하는 과제입니다. 시스템은 환자의 생체 신호를 주기적으로 측정하고, 해당 신호가 특정 한도를 초과할 때 경고 레벨을 설정합니다. 경고 레벨은 **Green, Yellow, Orange, Red**로 구분되며, 각 질병에 따라 경고 기준이 다릅니다. 이 시스템은 데이터베이스에서 환자 정보를 로드하고, 이후에는 파일에서 정보를 로드할 수 있도록 개선됩니다.

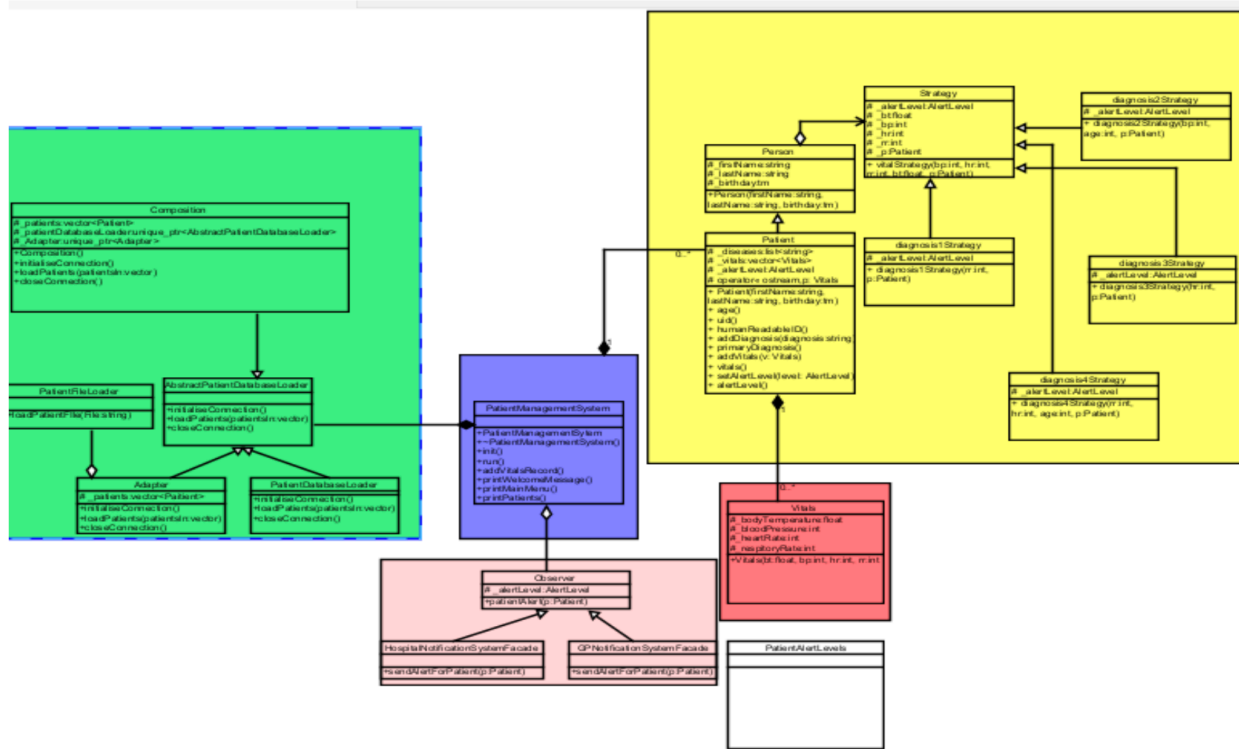


Figure1: Whole Design Structure

Output:

```

WELCOME TO HEALTHCO 4000
-----

Select an option:
1. List patients
2. Add vitals record
3. Quit
> 1
jh1074|02-10-1974|Jones, Henry|BonusEruptus|37.5,80,60,16;37.5,82,15,16
mm0580|24-05-1980|McCartney, Martha|AmoriaPhlebitis|
tg0548|15-05-1948|Taylor, George|MadZombieDisease|37.2,80,60,12
zd01100|01-01-2000|Zaius, Dr|ThreeStoogesSyndrome|
kk0890|19-08-1990|Kroker, Kif|BonusEruptus;MadZombieDisease|40.2,70,60,12;39.7,80,40,12
bs02115|02-02-2015|Baratheon, Shireen|ThreeStoogesSyndrome;BonusEruptus|

Select an option:
1. List patients
2. Add vitals record
3. Quit
> 2
Patients
jh1074|02-10-1974|Jones, Henry|BonusEruptus|37.5,80,60,16;37.5,82,15,16
mm0580|24-05-1980|McCartney, Martha|AmoriaPhlebitis|
tg0548|15-05-1948|Taylor, George|MadZombieDisease|37.2,80,60,12
zd01100|01-01-2000|Zaius, Dr|ThreeStoogesSyndrome|
kk0890|19-08-1990|Kroker, Kif|BonusEruptus;MadZombieDisease|40.2,70,60,12;39.7,80,40,12
bs02115|02-02-2015|Baratheon, Shireen|ThreeStoogesSyndrome;BonusEruptus|

Enter the patient ID to declare vitals for > jh1074
enter body temperature: 36.5
enter blood pressure: 88
enter heart rate: 60
enter respiratory rate: 55
Patient: Jones, Henry (jh1074)has an alert level: Red

This is an alert to the hospital:
Patient: Jones, Henry (jh1074) has a critical alert level

This is a notification to the GPs:
Patient: Jones, Henry (jh1074) should be followed up
    
```

두 과제 모두 C++을 활용하여 실시간 데이터 처리 및 사용자 상호작용을 다루고 있으며, 실제 상황에서 활용 가능한 시스템을 구축하는 데 중점을 두었습니다.

📌 프로젝트 GitHub: <https://github.com/Gitaebaechaos/portfolio/tree/main/C++>

Pepper Robot

- 작업기간 : 2023-07-03 ~ 2023-11-28

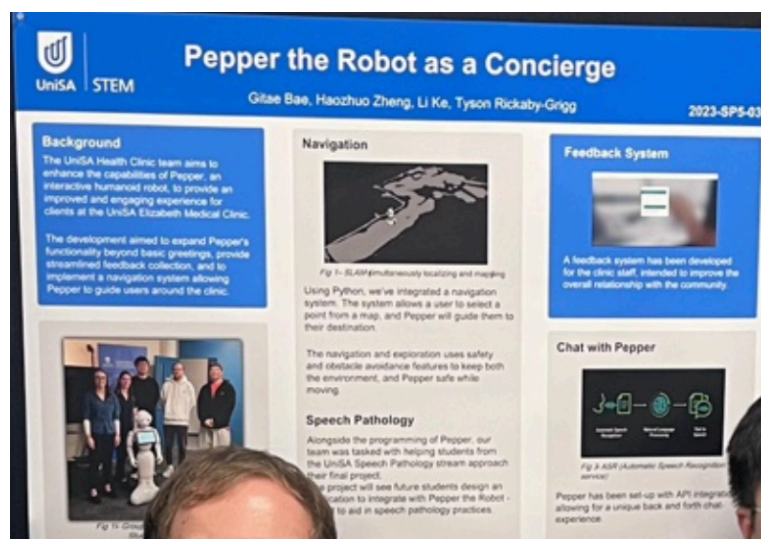
- 작업인원 : 4 명

- 작업 툴 : Choreographe

- 작품소개 : Choreographe를 사용해서 전 학기 그룹원들이 만들어놓은 프로젝트를 다음학기인 우리 그룹이 이어 받아서 페퍼로봇을 학습시키고 교육시키고 그거에 따른 여러가지 기능들을 추가해서 다양한 언어를 활용하여서 pepper robot을 발전시켰다.

본 프로젝트의 목표는 Unisa Health Clinic 내에서 Pepper라는 휴머노이드 로봇의 기능을 개선하여, 클라이언트들에게 더 인터랙티브하고 개인화된 효율적인 경험을 제공하는 것입니다. 주요 기능은 클라이언트에게 다이나믹하게 인사하고, 개인화된 대화와 관련된 정보를 제공하여 친근한 환경을 만드는 것입니다. 또한, 피드백 수집 시스템을 개선하고 사용자 친화적인 인터페이스를 구현하여 피드백을 관리하고 분석할 수 있도록 합니다. Pepper가 클리닉 내에서 특정 위치로 안전하고 효율적으로 안내할 수 있도록 내비게이션 기능도 향상시킵니다. 전반적으로 본 프로젝트는 Pepper를 클리닉 환경에서 더욱 효율적이고 다재다능하게 만들기 위해 최신 기술을 통합하는 것을 목표로 합니다.

자료:





- 📌 프로젝트 GitHub: <https://github.com/Gitaebacchaos/portfolio/tree/main/Pepper Robot>
- 📌 프로젝트 GitHub: <https://github.com/Gitaebacchaos/portfolio/tree/main/Pepper Robot2>

Cloud and Concurrent

- 작업기간 : 2023-07-07 ~ 2023-11-05
- 작업인원 : 1 명
- 작업 툴 : Cloud, java
- 작품소개 : Cloud와 Concurrent programming을 이해하기 위해 Uber와 유사한 시스템에 대해 리포트를 작성하고, 이를 바탕으로 자바를 이용해 코딩을 진행한 프로젝트입니다. 이 시스템은 클라우드 기반의 교통 서비스 시뮬레이션을 다루며, 동시 실행되는 예약 시스템을 구현하였습니다.

Assignment 1: Nüber (다큐먼트)

Nüber는 Über와 유사한 개인 운송 서비스로, 시스템 관리자, 운전자, 고객의 세 가지 사용자 역할을 포함한 클라우드 소싱 플랫폼을 개발하는 프로젝트입니다. 이 프로젝트는 AWS 클라우드 기반으로 전 세계적으로 확장 가능한 효율적인 애플리케이션을 설계하고, 개인 정보 보호, 데이터 저장 위치, 보안 및 저장 솔루션을 고려하여 시스템을 설계하는 것을 목표로 합니다.

Breakdown

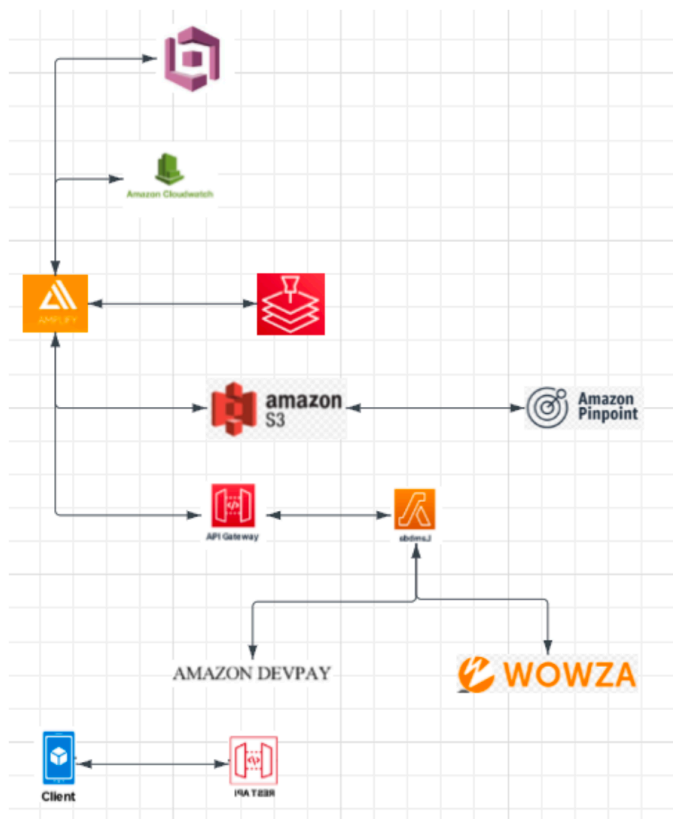


Figure 3: Cloud AWS Diagram

Assignment 2: Nüber (개발: 코딩)

두 번째 과제에서는 Nüber의 새로운 정부 규제 영향을 평가하기 위한 시뮬레이터를 개발합니다. 이 규제는 각 도시에 운전자가 활동할 수 있는 최대 수를 제한하고, 시뮬레이터를 통해 제한 값에 따른 영향을 분석합니다. 여러 예약이 동시에 처리될 수 있도록 하여, 프로그램은 예약이 완료되면 자연스럽게 종료됩니다.

Sample run without debug out

The following is an example run for

```
new Simulation(regions, 5, 10, 1000, false);

Creating Nuber Dispatch
Creating 2 regions
Creating Nuber region for South
Creating Nuber region for North
Done creating 2 regions
Active bookings: 10, pending: 6
Active bookings: 8, pending: 4
Active bookings: 2, pending: 1
Active bookings: 0, pending: 0
Simulation complete in 4040ms
```

🔗 프로젝트 GitHub: <https://github.com/Gitaebaechaos/portfolio/tree/main/Cloud and Concurrent>

Android

- 작업기간 : 2024-02-28 ~ 2024-06-28
- 작업인원 : 2 명
- 작업 툴 : android studio
- 작품소개 : Android Studio를 사용해 팀원과 함께 3단계에 걸친 작은 안드로이드 게임을 개발했다. 이 게임은 사용자가 화면을 터치해 장애물을 피하고 최대 거리를 이동하는 목표로, 친구들과 함께 즐기기 좋은 게임이다.

"Who Pays the Bill" 게임은 친구들과 함께 술자리에서 즐기기 좋은 게임으로, 장애물을 피하며 가능한 한 멀리 이동하는 것이 목표입니다. 화면에 다양한 장애물이 표시되며, 사용자는 화면을 터치하여 주인공을 조종하고 장애물을 피해야 합니다. 가장 높은 점수를 얻은 사람이 승리합니다. 이 문서는 게임의 개념, 메커니즘, 사용자 상호작용, 팀의 역할, 구현 개요를 제공합니다. 제공된 샘플 코드를 바탕으로 게임 프로토타입을 효과적으로 개발하고 개선할 수 있습니다.

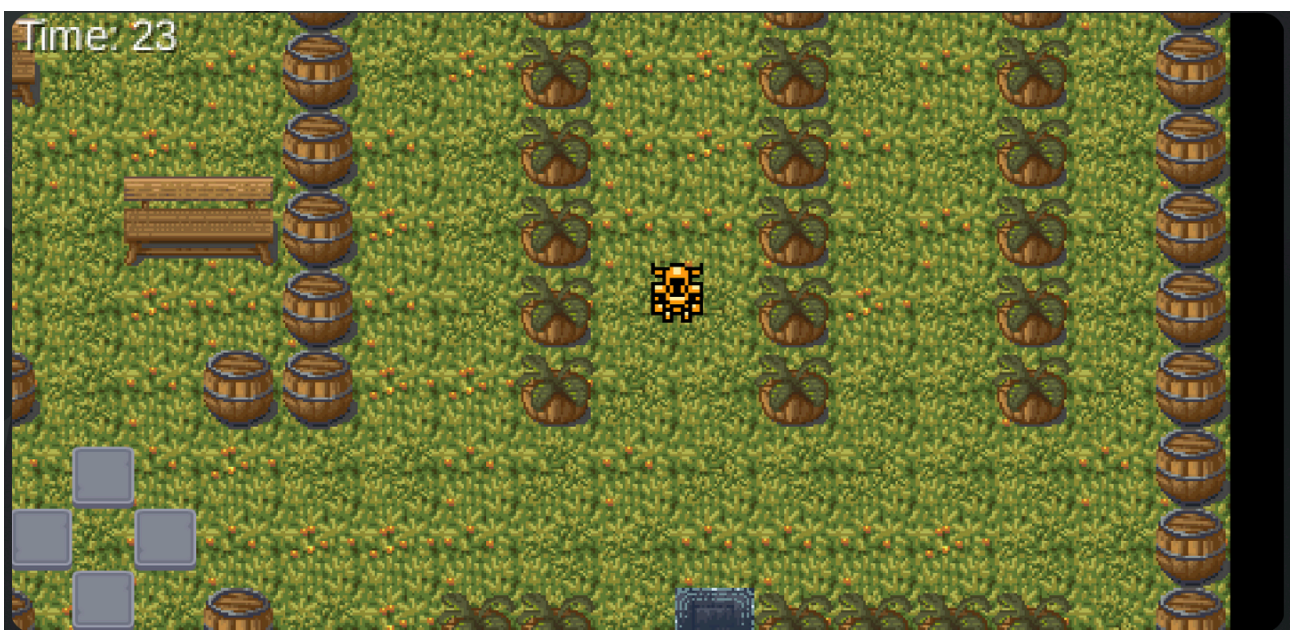
Stage1

1단계: 게임은 간단한 로딩 화면으로 시작된다. 목표는 장애물을 피하면서 설정된 거리를 도달하는 것이다. 설정된 거리를 달성하면 다음 단계로 넘어가며, 점수를 달성하지 못하면 게임이 종료된다.



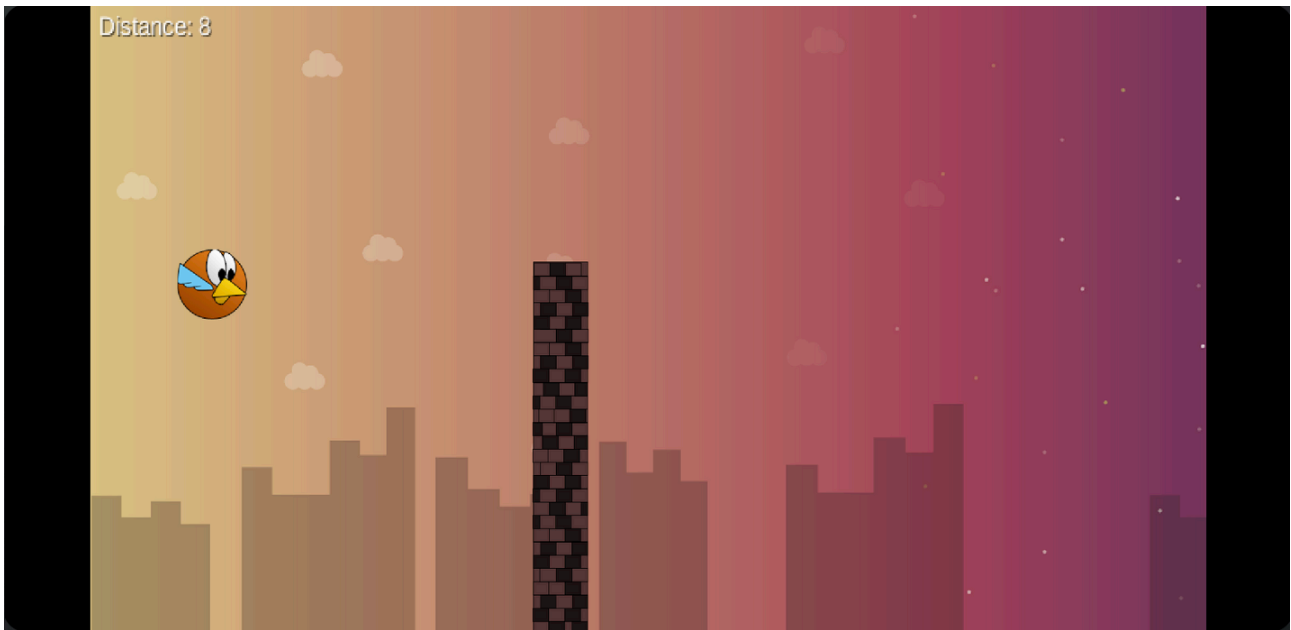
Stage2

2단계: 목표는 제한된 시간 내에 목표 지점에 도달하는 것이다. 점수를 달성하지 못하거나 제한 시간 내에 목표에 도달하지 못하면 게임이 종료된다.



Stage3

3단계: 목표는 장애물을 피하면서 설정된 거리를 도달하는 것이다. 50거리를 달성하면 게임이 성공적으로 끝나며, 장애물에 부딪히면 게임이 종료된다.



📌 프로젝트 GitHub: <https://github.com/Gitaebaechaos/portfolio/tree/main/Android>

Portfolio

최종:

작업기간 : 2021-07-01 ~ 2024-08-30 작업인원 : 1 명 작업 툴 : Eclipse, Xcode etc... 작품소개 : 다양한 프로젝트들을 깃허브에 등록해두었다. 너무 큰 파일들은 zip 파일로 올렸고 그 외 파일들은 기본 폴더로 올려두었다. 각 폴더에 README 파일로 각 프로젝트 간단한 설명을 해두었고 그 외 파일들은 작업했던 파일들이다.

📌 프로젝트 GitHub: <https://github.com/Gitaebaechaos/portfolio>